Pedidos360 · Plan de acción

Evaluación Parcial N° 1 · DSY1107 Desarrollo Cloud Native I · 11 de octubre de 2026

El sistema funciona de punta a punta, pero hoy no usa las tecnologías que la pauta evalúa: no integra MSAL y el gateway valida un JWT propio en lugar del token del IDaaS. El plan tiene cinco fases. Las tres primeras cubren el 100 % de la rúbrica; las dos últimas cubren requisitos del encargo que la rúbrica no puntúa directamente (pruebas, AWS y base de datos cloud).

Dónde estamos

Indicador	Peso	Qué pide la pauta	Qué tiene el proyecto hoy
MSAL en Angular	60 %	MSAL integrado, MsalGuard y MsalInterceptor operativos, tokens del IDaaS, roles y scopes leídos de los claims.	Guard e interceptor propios con un JWT interno en localStorage. MSAL no está instalado. Sin roles ni scopes.
Validación en el BFF	40 %	Validar issuer, audience, firma y vigencia contra el IDaaS; autorización por rol; códigos de error adecuados.	Valida firma y vigencia de un JWT propio (HS256). No valida issuer ni audience, no hay roles, y los microservicios aceptan llamadas directas sin token.

Decisión previa

Con MSAL, el login local y los botones de Google, Facebook y Discord dejan de pasar por el backend. **Recomendación: eliminarlos.** El registro con correo queda cubierto por el flujo de registro de Entra External ID, y Google o Facebook se pueden federar desde el mismo tenant si se quieren mantener. Mantener ambos sistemas en paralelo obligaría al gateway a aceptar dos tipos de token y debilita justo lo que la pauta evalúa.

Consecuencia: los usuarios, carritos y pedidos actuales no se conservan, porque el identificador de usuario pasa a ser el claim `oid` de Entra.

Fase 1 · Configurar Entra

Responsable: el equipo, en el portal de Entra · Bloquea las pruebas de las fases 2 y 3

1. En el registro de la aplicación, agregar la plataforma **SPA** con redirect URI `http://localhost:4200`.
2. En **Exponer una API**, definir el Application ID URI y un scope, por ejemplo `api://<client-id>/access_as_user`.
3. En **Roles de aplicación**, crear `admin` y `cliente` y asignarlos a los usuarios de prueba.
4. En **Configuración de token**, agregar el claim opcional `email` al access token.
5. Fijar la versión del access token en v2 (`requestedAccessTokenVersion: 2` en el manifiesto).

Fase 2 · MSAL en Angular

Responsable: código del frontend · 60 % de la nota

1. Instalar `@azure/msal-browser` y `@azure/msal-angular`.
2. Configurar MSAL en `app.config.ts` con el client ID, la authority del tenant y el scope de la API.
3. Reemplazar `auth.guard.ts` por `MsalGuard` en las rutas privadas.
4. Reemplazar `auth.interceptor.ts` por `MsalInterceptor`, con el gateway como recurso protegido.
5. Reescribir `auth.service.ts` para que login y logout usen MSAL y el usuario, los roles y los scopes se lean de los claims.
6. Mostrar u ocultar las opciones de administrador según el rol.
7. Quitar el modal de login y registro local y la ruta `auth/callback`.

Fase 3 · Validación en el gateway

Responsable: código del backend · 40 % de la nota

1. Agregar `spring-boot-starter-oauth2-resource-server` al gateway y eliminar `JwtGatewayFilter`.
2. Configurar el `issuer-uri` del tenant: con eso se validan la firma (JWKS de Microsoft), el issuer y la vigencia.
3. Agregar un validador del claim `aud` contra el client ID de la API.
4. Mapear el claim `roles` a autoridades y restringir las rutas de administración al rol `admin`.
5. Responder 401 (sin token o token inválido) y 403 (sin el rol requerido) con un JSON de error consistente.
6. Seguir enviando `X-User-Id` a los microservicios, pero tomado del claim `oid`, y descartar cualquier cabecera `X-User-*` que venga del cliente.
7. Quitar los puertos publicados de los microservicios y las bases en `docker-compose.yml`, para que solo el gateway sea accesible.
8. En `msvc-usuario`, eliminar el login local, el OAuth propio y la emisión del JWT interno.

Fase 4 · Pruebas básicas

Responsable: código del backend · Requisito del encargo

1. Un test de arranque de contexto por microservicio.
2. Tests del gateway: sin token responde 401, con token válido responde 200, con token sin rol en ruta de administración responde 403.
3. Tests de controlador para carrito y pago.

Fase 5 · Despliegue en AWS

Responsable: el equipo, en la consola de AWS · Requisito del encargo

1. **RDS PostgreSQL**: una instancia con las seis bases, y apuntar las variables `*_DB_URL` a ella.
2. **EC2**: una instancia con Docker que levante el compose del backend.
3. **AWS API Gateway**: una HTTP API con autorizador JWT (mismo issuer y audience que el gateway) que enrute a la EC2.
4. Cambiar `API_BASE_URL` del frontend y el CORS a la URL del API Gateway, y agregar esa URL como redirect URI en Entra si se publica el frontend.

Orden de trabajo

Paso	Fase	Por qué en este orden
1	Fase 1 · Entra	Sin el scope, los roles y la plataforma SPA no se puede probar el login ni la validación.
2	Fases 3 y 2 en paralelo	Gateway y frontend se prueban uno contra el otro; cubren toda la rúbrica.
3	Fase 4 · Pruebas	Fijan el comportamiento 401/403 antes de mover el sistema a la nube.
4	Fase 5 · AWS	Se despliega lo que ya funciona en local; solo cambian URLs y variables de entorno.

Cómo sabremos que cumplimos

1. Iniciar y cerrar sesión con una cuenta del tenant funciona desde el frontend, y las rutas privadas redirigen al login.
2. Las llamadas al gateway llevan el access token de Entra, y la interfaz muestra los roles leídos del token.
3. El gateway responde 401 sin token, con token expirado o con audience distinta, y 403 a un usuario sin rol de administrador.
4. Ningún microservicio responde si se le llama directamente, sin pasar por el gateway.
5. Las pruebas de todos los servicios pasan y el frontend compila sin errores.
6. El mismo flujo funciona contra la URL del AWS API Gateway, con los datos en RDS.